



Batchflow Project Specification v2

Turn data into progress.

Implementation profile: Astro + SolidJS + TypeScript + Neon Postgres + Netlify

Batchflow is an asynchronous CSV import tool. Users upload a source file, map columns, validate sample rows, run an import and inspect row-level failures. Unlike the other projects, Batchflow requires an explicit background-processing design so long-running imports do not block normal HTTP requests.

Purpose of this document

This specification translates the product PRD into an implementation plan. It defines the application architecture, database shape, server interfaces, UI modules, service integrations, testing strategy, deployment model and delivery criteria.

1. Product and Functional Scope

User roles

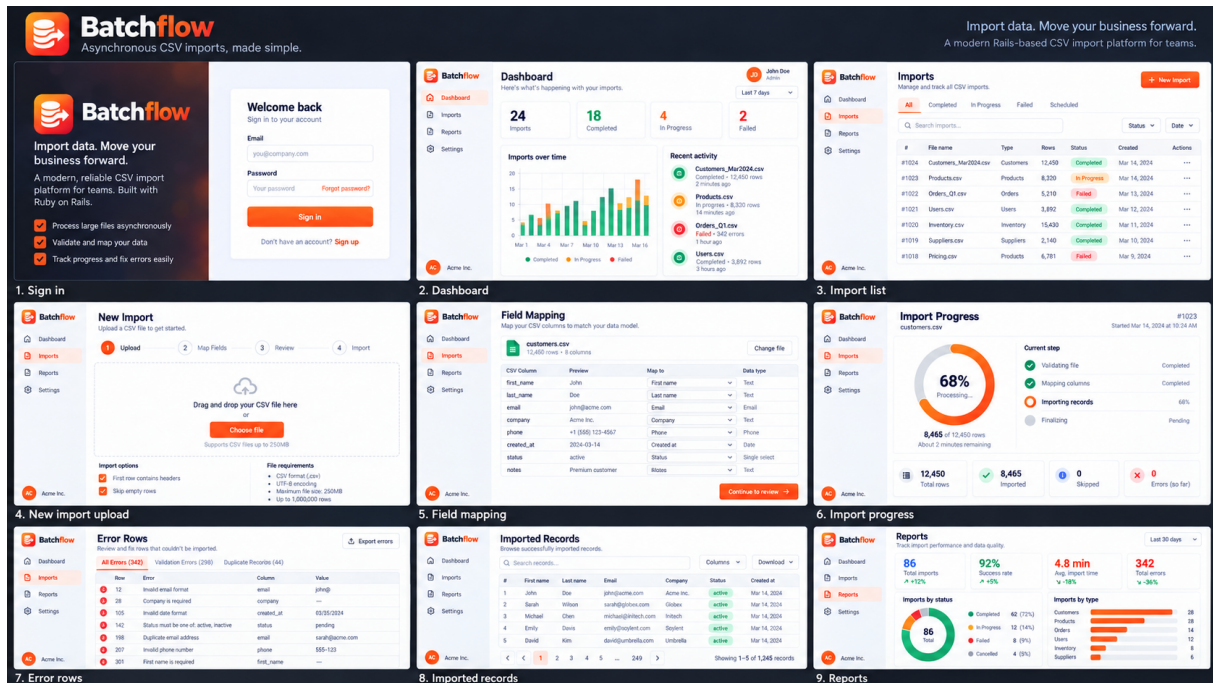
User	Upload CSV files, configure mappings, run imports and inspect results.
Administrator	Inspect all imports, retention settings and system failures.

Required screens

- Sign in
- Dashboard
- Import list
- New import
- Field mapping
- Validation preview
- Import progress
- Error rows
- Imported records
- Reports

Scope discipline

Build the primary workflow first. Optional analytics, advanced filters and secondary settings should only be added after the core workflow, permissions, tests and deployment are stable.



2. System Architecture

Target architecture

Browser -> Cloudflare DNS -> Netlify -> Astro server routes/pages -> SolidJS islands -> TypeScript service layer -> parameterized SQL -> Neon Postgres. Neon Auth provides authentication. R2, Resend and other services are added only where the project requires them.

Architecture principles

- Use Astro for routing, SSR, API/server endpoints and page composition.
- Use SolidJS only for interaction-heavy islands rather than turning the product into an unnecessary SPA.
- Keep server-only code outside the client bundle.
- Use direct parameterized SQL; centralize queries by domain to avoid SQL scattered through UI components.
- Validate every external mutation with Zod on the server.
- Authorization is enforced server-side even when controls are hidden in the UI.
- Keep infrastructure minimal; do not add services without a concrete product need.

Repository structure

```
src/ components/ layouts/ pages/ pages/api/ lib/ auth/ db/ validation/ services/ features/ styles/ tests/  
unit/ e2e/ public/ netlify/ functions/ (only where needed) .github/workflows/
```

3. Technology Stack

Application framework	Astro
Interactive UI	SolidJS
Language	TypeScript
Build tooling	Vite (through Astro)
Runtime	Node.js
Styling	Tailwind CSS
Database	Neon Postgres
Authentication	Neon Auth
Database access	Direct parameterized SQL - no ORM
Validation	Zod
Object storage	Cloudflare R2 where files are required
Email	Resend where transactional mail is required
Testing	Vitest + Playwright
Code quality	ESLint + Prettier
Monitoring	Sentry
CI/CD	GitHub Actions
Hosting	Netlify
DNS / edge	Cloudflare DNS
TLS	Netlify-managed certificate; Cloudflare proxy optional with Full (strict)

Node.js usage

Node.js is the server/runtime and tooling environment. Pin a current LTS version in the repository so local development, CI and Netlify use a consistent runtime.

Direct SQL conventions

- Use the Neon serverless/Postgres driver appropriate for the chosen Astro runtime.
- All values are passed as parameters; never construct SQL with untrusted string interpolation.
- Keep migrations as versioned SQL files in the repository.
- Wrap multi-step state changes in database transactions.
- Create indexes for foreign keys and fields used heavily by filtering/search.

4. Data Model

Core tables

users	id, email, name, role
imports	id, user_id, source_r2_key, filename, status, total_rows, processed_rows, success_rows, failed_rows, started_at, completed_at
import_mappings	id, import_id, source_column, target_field, transform_json
import_rows	id, import_id, row_number, status, target_record_id
import_errors	id, import_id, row_number, field, source_value, error_code, error_message
imported_records	id, import_id, payload_json, created_at

Migration strategy

Store ordered SQL migration files in source control. Each migration must be forward-only for the portfolio MVP. Seed scripts should create realistic demo data without embedding production credentials.

Data integrity

- Use NOT NULL constraints for required fields.
- Use foreign keys for relational ownership.
- Use CHECK constraints for stable enums/statuses where practical.
- Use unique constraints for natural keys such as API keys, feature keys or emails where applicable.
- Use timestampz for timestamps.

5. Server Routes and UI Modules

Primary server interfaces

- POST /api/uploads/presign - upload CSV to R2
- POST /imports - create import metadata
- POST /imports/:id/mappings - save mapping
- POST /imports/:id/validate - validate sample
- POST /imports/:id/start - enqueue/background start
- GET /imports/:id/status - progress polling
- GET /imports/:id/errors.csv - error export

Primary UI components / Solid islands

- CSVUploader
- ColumnMapper
- ValidationPreview
- ImportProgress
- ErrorTable
- ImportSummaryCards
- ImportHistoryTable

Implementation pattern

Pages should render useful content on the server. Solid islands handle stateful widgets such as filters, forms, calendars, progress displays, inspectors and interactive tables. Server mutations should return structured success/error results that map cleanly to form states.

6. Business Rules, Security and Integrations

Project-specific rules

- Only CSV files within configured limits are accepted.
- Source files are stored in R2; import metadata and results are stored in Neon.
- Import processing runs outside the initial HTTP upload request.
- Each row is validated independently with Zod.
- One malformed row must not fail the entire import unless the file itself is unreadable.
- Progress counters are updated periodically, not necessarily on every row.

External services

Neon Auth	Identity.
Cloudflare R2	Uploaded CSV source files and generated error exports if desired.
Netlify Background Functions / Cloudflare Queues	Background import execution - choose one implementation for MVP.
Sentry	Import exceptions and failed background work.

Security baseline

- Rely on Neon Auth/session state server-side for identity.
- Validate authorization on every protected server route.
- Use Zod for request payload validation.
- Use secure, HTTP-only cookies according to the Neon Auth integration.
- Rate-limit abuse-sensitive public endpoints where practical.
- Never expose database, R2, Resend or Sentry server secrets in browser code.
- Escape/sanitize user-controlled text before rendering.
- Log errors to Sentry without logging secrets or sensitive payloads.

7. Testing and Quality

- Vitest: CSV parsing, mappings, Zod row schemas, duplicate handling and error aggregation.
- Playwright: upload, map fields, validate, start import, watch progress and download errors.
- Load sanity test with a representative multi-thousand-row CSV.

CI pipeline

On each pull request / push: 1. npm ci 2. TypeScript typecheck 3. ESLint 4. Prettier check 5. Vitest 6. Build Astro application 7. Playwright on selected branches or against preview deployment

Quality gate

A feature is not considered complete until its server-side validation, authorization path and primary failure case have automated coverage.

8. Deployment and Operations

Netlify

- Connect the GitHub repository to Netlify.
- Use deploy previews for pull requests.
- Configure Astro's Netlify adapter/runtime as required by the final app mode.
- Store all secrets in Netlify environment variables.
- Use Netlify Functions/Background Functions only where the product requires them.
- Expose a lightweight health route where useful.

Cloudflare DNS / TLS

Point the chosen custom domain/subdomain to Netlify through Cloudflare DNS. Netlify can provision the origin/site certificate. If Cloudflare proxying is enabled, use Full (strict) so Cloudflare validates the Netlify certificate.

Environment variables

- DATABASE_URL
- NEON_AUTH_*
- R2_ACCOUNT_ID
- R2_ACCESS_KEY_ID
- R2_SECRET_ACCESS_KEY
- R2_BUCKET
- SENTRY_DSN
- QUEUE/BACKGROUND_* if required

Observability

Configure Sentry for production and preview environments. Capture unhandled server errors and meaningful client failures. Add release/environment tags so a deployed error can be traced to a commit.

9. Implementation Sequence

- 1. Bootstrap repository: Astro, Solid, TypeScript, Tailwind, ESLint, Prettier, Vitest and Playwright.
- 2. Configure Neon Postgres and versioned SQL migrations.
- 3. Integrate Neon Auth and implement authorization helpers.
- 4. Build the core data model and seed/demo data.
- 5. Implement one complete end-to-end workflow before secondary screens.
- 6. Add project-specific external services such as R2 or Resend.
- 7. Add Solid islands only where interaction materially benefits from them.
- 8. Add Vitest coverage for domain logic and Playwright coverage for primary user journeys.
- 9. Configure GitHub Actions, Netlify previews and Sentry.
- 10. Polish README, screenshots, demo account/data and known limitations.

Definition of Done

- CSV upload works.
- Mapping and validation preview work.
- Processing is asynchronous.
- Valid rows import and invalid rows are reported.
- Progress is visible.
- Error export works.
- Tests pass.
- Netlify deployment is healthy.

Portfolio requirement

The repository README should include the project banner/logo, one-sentence value proposition, live demo link, screenshots, feature list, architecture summary, tech stack, local setup, environment-variable template, test commands, known limitations and license.